This example demonstrates two different types of functions. The first is a module-based function (`IO.puts/1`) and the second is an anonymous function that we've assigned to the `adder` variable.

This looks a bit foreign right now, but the next chapter will clear it all up!

Wrapping Up

Ruby is a language with joy at its core. The philosophy that pins the foundation of the language's design also drives the community and culture that has evolved around Ruby. This has allowed Ruby to remain relevant and continually evolve over the years. It has been and remains a great language.

When looking for a new language, these qualities shouldn't be sacrificed: joy, pragmatism, healthy community, and continual innovation. Elixir delivers on these qualities, and it has emerged into a language loved by many. Deciding to learn a new language is a big decision, but Elixir is worth the effort.

Elixir provides a world-class technical foundation for all types of applications. Web development is a primary use case, but hardware and machine learning are also handled really well. Elixir's focus on easy parallelism and fault tolerance gives it a huge advantage over other languages.

Elixir doesn't exist by itself—it stands on the shoulders of giants that have been in development for decades. Erlang, OTP, and the BEAM help Elixir deliver a world-class development and production experience. But it's done in such a way that you don't need to worry about the foundations that it's built on. You can write and ship Elixir but still benefit from the rock-solid BEAM runtime.

You wrote some of your first lines of Elixir code in this chapter. In the next chapter, we'll dive deeper into the Elixir language. You'll learn about basic syntax, modules, functions, lists, and structs. Along the way, these concepts will be compared to equivalent concepts in Ruby. Let's get to it!